

CLAUDIA MARIA LOPEZ BOMBIN

GONZALO VELASCO LUCAS

NOVIEMBRE, 2025

PROYECTO FINAL

**O SEA, ¿CÓMO QUE CONFIGURAR ESTE PROYECTO TIENE
MÁS SCRIPTS QUE ERRORES EN MI PRIMER CÓDIGO EN
BASH?**

FUNDAMENTOS DE LOS SISTEMAS OPERATIVOS

PROFESOR

GRUPO XXXXX

ÍNDICE

INTRODUCCIÓN3

INSTALACIÓN Y CONFIGURACIÓN DE ALPINE LINUX5

INSTALACION DE PAQUETES6

DESPILEGUE DEL PROYECTO FLASK13

PROCESAMIENTO DE DATOS GEOGRAFICOS.....23

PETICIONES AUTOMATIZADAS.....26

ANALISIS DE RESULTADOS Y DISCUSION.....30

AUTORÍA.....36

BIBLIOGRAFÍA.....12

INTRODUCCIÓN

Este documento, realizado por Claudia Maria Lopez Bombin y Gonzalo Velasco Lucas – estudiantes de 2B del Grado en Ingeniería Matemática e Inteligencia Artificial en la Universidad Pontificia Comillas – constituye la memoria técnica del proyecto de la asignatura Fundamentos de Sistemas Operativos, titulado "Despliegue Integral de una Aplicación Flask con Procesamiento Distribuido en Entorno Alpine Linux".

Los fundamentos de los sistemas operativos Linux que hoy sustentan nuestros servidores y aplicaciones en la nube encuentran sus raíces en el movimiento de software libre iniciado por Richard Stallman y el desarrollo del kernel Linux por Linus Torvalds en 1991. Como destacó Torvalds en su anuncio original, "estoy haciendo un sistema operativo (gratis) (solo un hobby, no será grande ni profesional como GNU)" (Moody & Torvalds, 2015), estableciendo sin saberlo las bases de lo que sería una revolución en el mundo de la computación.

Lo que comenzó como un proyecto personal se ha transformado en el pilar fundamental que soporta desde supercomputadores hasta dispositivos embebidos, demostrando que la colaboración abierta puede generar sistemas robustos y confiables. La filosofía UNIX de "hacer una sola cosa y hacerla bien" (McIlroy, 1978) permea todo el ecosistema de herramientas que hemos utilizado en este proyecto, desde los scripts en bash hasta la gestión de procesos.

Los sistemas de virtualización como VirtualBox -precisamente la plataforma que empleamos para desplegar Alpine Linux- permiten aislar entornos de ejecución y reproducir configuraciones complejas. Como señala Smith y Nair en su trabajo fundamental sobre virtualización, "la virtualización permite ejecutar múltiples sistemas operativos como invitados en un mismo equipo físico" (Smith & Nair, 2005). Desde el desarrollo de aplicaciones hasta el despliegue en producción, los entornos virtualizados trabajan incansablemente en segundo plano.

La transformación de comandos de terminal en aplicaciones web funcionales ejemplifica cómo los fundamentos de sistemas operativos pueden convertirse en facilitadores esenciales de servicios accesibles vía web, demostrando que el conocimiento de bajo nivel puede traducirse en funcionalidades de alto nivel para usuarios finales.

En el contexto de la administración de sistemas y el despliegue de aplicaciones se desarrolla este proyecto, cuyo objetivo principal es implementar un sistema completo de despliegue automático de aplicaciones Flask mediante scripts en bash, demostrando la integración entre sistemas operativos, entornos virtuales Python y aplicaciones web, y afianzando los fundamentos técnicos que sustentan el desarrollo y despliegue de software moderno.

Como objetivos secundarios de este proyecto encontramos:

- Profundizar en el desarrollo de scripts bash para administración de sistemas, optimizando la automatización de despliegues, instalación de dependencias y configuración de entornos Linux en distribuciones minimalistas como Alpine.
- Diseñar e implementar pipelines de procesamiento de datos eficientes, manejando datasets masivos de localizaciones geográficas (Gowalla) y optimizando el rendimiento en entornos con recursos limitados.
- Construir una aplicación web completa con Flask que integre múltiples servicios especializados: machine learning, análisis estadístico, generación de mapas interactivos y procesamiento paralelo.

- Implementar mecanismos exhaustivos de gestión de entornos virtuales Python, garantizando la reproducibilidad del proyecto mediante el uso de virtualenvs y gestión precisa de dependencias con requirements.txt.
- Garantizar la fiabilidad del sistema mediante scripts de verificación automática que validen la instalación de paquetes, la correcta configuración del entorno y la funcionalidad de todos los servicios implementados.
- Integrar múltiples tecnologías y librerías especializadas (scikit-learn para machine learning, folium para mapas, multiprocessing para computación paralela, requests para APIs) en una arquitectura de software coherente y mantenible.
- Dominar las técnicas de virtualización con VirtualBox/UTM, configurando redes adaptadas a diferentes escenarios (NAT, adaptador puente) y optimizando los recursos asignados a la máquina virtual.
- Fomentar el trabajo colaborativo en administración de sistemas mediante la distribución de tareas de configuración, desarrollo de scripts complementarios y integración de módulos desarrollados de forma independiente.

El proyecto consiste en la implementación de un sistema de despliegue integral compuesto por multiples scripts y modulos principales:

1. **apartado1_Claudia_Gonzalo.sh:** Script de instalación y configuración del sistema Alpine Linux que automatiza la instalación de todos los paquetes necesarios (python3, git, sudo, gcc, herramientas de desarrollo) implementando verificaciones inteligentes de dependencias ya instaladas.
2. **apartado_despliegue_Bash_Claudia_Gonzalo.sh:** Pipeline de despliegue automático que realiza el clonado del repositorio Git, creación del entorno virtual Python, instalación de dependencias desde requirements.txt y puesta en marcha de la aplicación Flask con todos sus servicios.
3. **script_procesamiento_datos_Claudia_Gonzalo.sh:** Sistema de procesamiento de datasets Gowalla que descarga, descomprime, filtra y analiza millones de check-ins geográficos, generando tanto estadísticas agregadas como archivos preparados para visualización.
4. **Módulos Python especializados:**
 - o **compare_execution.py:** Implementación de algoritmos de multiplicación matricial secuencial y paralela para benchmarking de rendimiento computacional
 - o **topn_selection_Claudia_Gonzalo.py:** Sistema de análisis de usuarios que calcula rankings basados en frecuencia de interacciones
 - o **estadisticas_python_Claudia_Gonzalo.py:** Herramienta de análisis estadístico que replica funcionalidades de línea de comandos usando solo librerías estándar
 - o **peticiones_automaticas_Claudia_Gonzalo.py:** Cliente automatizado para testing y interacción programática con los servicios Flask

El sistema integra múltiples capas tecnológicas, desde la configuración a bajo nivel del sistema operativo hasta servicios web de alto nivel, demostrando la aplicación práctica de los fundamentos de sistemas operativos a problemas reales de despliegue de aplicaciones y procesamiento distribuido. La arquitectura resultante sirve como ejemplo completo de pipeline DevOps para aplicaciones data-intensive.

INSTALACIÓN Y CONFIGURACIÓN DE ALPINE LINUX

1. Proceso de instalación

Para crear nuestra máquina virtual de Alpine Linux hemos debido descargar una imagen ISO que se nos fue proporcionada en el siguiente enlace <https://alpinelinux.org/downloads/>

Mediante la imagen, VirtualBox es capaz de crear la máquina virtual a la que le añadimos una configuración específica.

2. Configuración inicial del sistema

- Memoria base: 2048 MB (2 GB)
- Procesadores: 2 CPUs
- Tamaño del disco: 30 GB

Finalmente eliminamos la imagen ISO para ahorrar memoria

3. Creación de usuarios en la máquina

Al arrancar, accedemos como root y *ejecutamos* setup-alpine, que nos guía paso a paso por la configuración inicial del sistema. En este proceso se selecciona el idioma del teclado, el nombre del sistema entre otras cosas. Después, se establece la contraseña del usuario root, se ajusta la zona horaria, se elige el cliente NTP y se crea un usuario nuevo llamado alumnoimat con contraseña alumno.IMAT. También se selecciona el servidor SSH que permitirá la conexión remota. A partir de este punto, ya podemos acceder al sistema usando el nuevo usuario creado y continuar con la configuración y uso normal de Alpine Linux.

INSTALACION DE PAQUETES

SCRIPT DE INSTALACION (apartado1_Claudia_Gonzalo.sh)

El script `apartado1_Claudia_Gonzalo.sh` constituye el componente fundamental para la preparación del entorno Alpine Linux, actuando como sistema automatizado de provisioning que garantiza la disponibilidad de todas las dependencias necesarias para el despliegue del proyecto Flask. Su diseño responde a la necesidad de establecer una base sólida y reproducible sobre la cual construir el resto de la infraestructura del proyecto. La arquitectura del script se fundamenta en principios de robustez y verificación proactiva, implementando mecanismos que previenen errores comunes en entornos minimalistas como Alpine Linux. Su ejecución exitosa es un prerequisite crítico para todas las fases subsiguientes del proyecto.

Comandos Utilizados para Verificar e Instalar Paquetes

Gestión de Repositorios y Actualización del Sistema



```
1 echo ""
2 echo "Paso 1: Actualizando repositorios de Alpine..."
3 apk update
```

Este comando inicial sincroniza los índices de paquetes disponibles con los repositorios configurados en el sistema. En el contexto de Alpine Linux, esta operación es esencial debido a la naturaleza rolling-release de la distribución, donde los paquetes se actualizan frecuentemente y las dependencias pueden cambiar entre ejecuciones.

Verificación de Integridad del Repositorio Community



```
1 echo ""
2 echo "Paso 2: Verificando repositorio community..."
3 # El repositorio community debe estar habilitado para algunos paquetes
4 if ! grep -q "^[^#]*community" /etc/apk/repositories; then
5     echo "ERROR: Repositorio community no está habilitado"
6     echo "Para solucionar esto:"
7     echo "1. Edita el archivo: nano /etc/apk/repositories"
8     echo "2. Descomenta la línea que contiene 'community' (elimina el #)"
9     echo "3. Guarda el archivo y ejecuta este script nuevamente"
10    exit 1
11 else
12     echo "Repositorio community está habilitado"
13 fi
```

La verificación del repositorio community representa un punto crítico de validación, ya que varios paquetes esenciales (especialmente sudo y herramientas de desarrollo) residen exclusivamente en este repositorio. El patrón de expresión regular `^[^#].*community` asegura que:

- La línea no esté comentada (no comienza con #)
- Contenga la palabra "community" en cualquier posición
- Esté activa en la configuración actual del sistema

Mecanismo de Instalación Condicional por Paquete

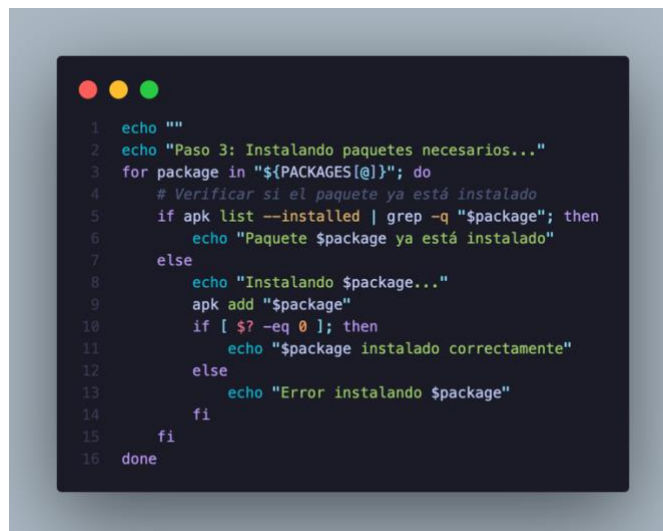


Para cada paquete en la lista definida, se ejecuta el comando de instalación estándar de Alpine Package Keeper (apk). La elección de instalar paquetes individualmente en lugar de en lote permite:

- Aislamiento de fallos: Un error en un paquete no detiene toda la instalación
- Logging granular: Seguimiento preciso del estado de cada componente
- Recuperación elegante: Reintentos selectivos en caso de fallos de red

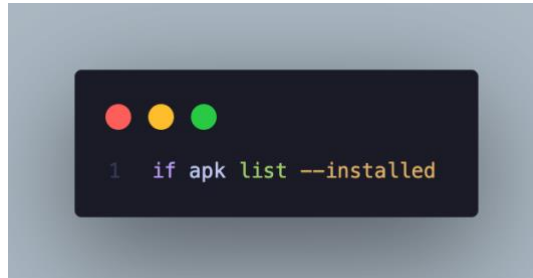
Lógica de Verificación de Paquetes Existentes

Sistema de Detección de Paquetes Instalados



Esta construcción implementa un mecanismo de idempotencia que garantiza que el script pueda ejecutarse múltiples veces sin efectos secundarios negativos. La lógica se descompone en:

1. Obtención del inventario del sistema:



Genera una lista completa de todos los paquetes actualmente instalados en el sistema, incluyendo información de versión y repositorio.

2. Búsqueda eficiente con grep:



La opción -q (quiet) permite una verificación silenciosa que solo retorna un código de estado, optimizando el rendimiento y evitando output innecesario.

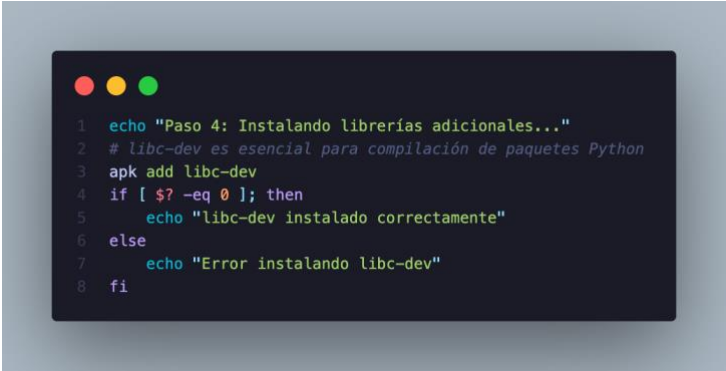
3. Evaluación condicional

Si el paquete existe (grep retorna 0): Se omite la instalación y se notifica al usuario

Si el paquete no existe (grep retorna 1): Se procede con la instalación

Manejo de Casos Especiales: libc-dev

El script trata la librería libc-dev como un caso especial mediante una instalación directa sin verificación previa, ya que nos ha reportado problemas previos al intentar instalarlo junto al resto de paquetes:



```

1  echo "Paso 4: Instalando librerías adicionales..."
2  # libc-dev es esencial para compilación de paquetes Python
3  apk add libc-dev
4  if [ $? -eq 0 ]; then
5      echo "libc-dev instalado correctamente"
6  else
7      echo "Error instalando libc-dev"
8  fi

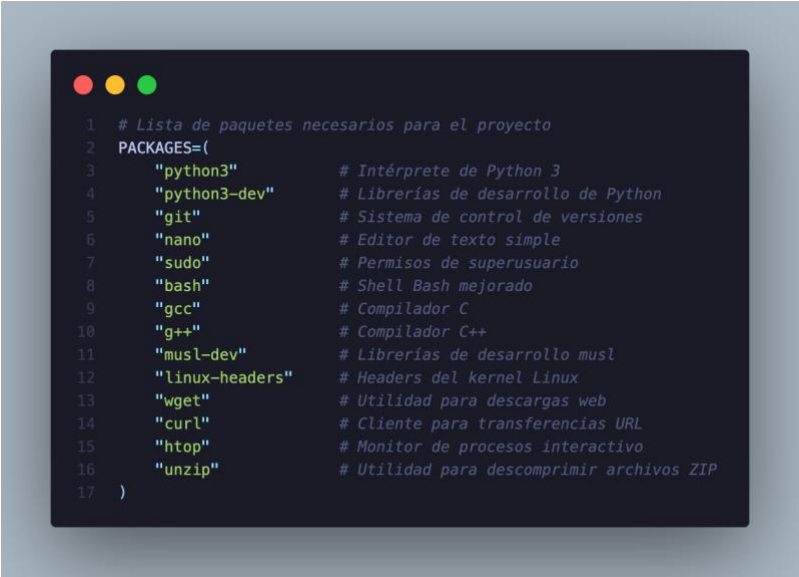
```

Esta decisión de diseño responde a consideraciones técnicas específicas:

- Dependencia implícita: libc-dev es requerida por numerosos paquetes de compilación pero puede no aparecer como instalada explícitamente
- Naturaleza fundamental: Constituye un componente base del toolchain de desarrollo
- Tolerancia a reinstalación: Su reinstalación no genera conflictos ni efectos adversos

Array de Paquetes y Estructura de Datos

La definición de paquetes utiliza un array Bash para encapsular la configuración:



```

1  # Lista de paquetes necesarios para el proyecto
2  PACKAGES=(
3      "python3"           # Intérprete de Python 3
4      "python3-dev"       # Librerías de desarrollo de Python
5      "git"               # Sistema de control de versiones
6      "nano"              # Editor de texto simple
7      "sudo"              # Permisos de superusuario
8      "bash"              # Shell Bash mejorado
9      "gcc"               # Compilador C
10     "g++"               # Compilador C++
11     "musl-dev"           # Librerías de desarrollo musl
12     "linux-headers"      # Headers del kernel Linux
13     "wget"               # Utilidad para descargas web
14     "curl"               # Cliente para transferencias URL
15     "htop"               # Monitor de procesos interactivo
16     "unzip"              # Utilidad para descomprimir archivos ZIP
17 )

```

Cada entrada incluye un comentario que documenta su propósito específico dentro del ecosistema del proyecto, facilitando el mantenimiento y la comprensión de las dependencias.

Sistema de Reporting y Logging

El script implementa un sistema de feedback visual comprehensivo mediante indicadores textuales claros:

- Mensajes de estado contextuales: Proporcionan contexto sobre la operación en curso
- Separadores visuales: Mejoran la legibilidad durante ejecución interactiva
- Códigos de retorno explícitos: `$? -eq 0` para verificación de éxito/error

El flujo de ejecución sigue una estructura lógica de cuatro fases:

1. Preparación: Actualización de repositorios y verificación de configuración
2. Validación: Comprobación del repositorio community (requisito crítico)
3. Instalación: Procesamiento iterativo de la lista de paquetes
4. Finalización: Instalación de componentes especiales y resumen final

Esta aproximación garantiza que el usuario tenga visibilidad completa del progreso de la instalación y pueda identificar rápidamente cualquier problema que surja durante la ejecución.

La combinación de estas estrategias—verificación preventiva, instalación condicional, y reporting comprehensivo—convierte al script en una herramienta robusta y confiable para la preparación del entorno de desarrollo, estableciendo las bases técnicas sobre las cuales se construye el resto de la infraestructura del proyecto Flask.

CONFIGURACIÓN DE PERMISOS SUDO

La configuración de permisos sudo constituye el componente fundamental para la transición desde el entorno de administración privilegiado (root) hacia un contexto de desarrollo seguro y controlado bajo el usuario `alumnoimat`. Esta transición responde a la necesidad de establecer un equilibrio entre la capacidad operativa necesaria para el despliegue del proyecto y las mejores prácticas de seguridad en entornos Linux. La arquitectura de esta configuración se fundamenta en principios de control de acceso granular y verificación sistemática, implementando mecanismos que previenen tanto la elevación indiscriminada de privilegios como las restricciones excesivas que impedirían el desarrollo fluido del proyecto.

Modificación del archivo `/etc/apk/repositories`

La configuración previa del repositorio `community` en `/etc/apk/repositories` constituye un prerequisite fundamental para la instalación del paquete `sudo` en Alpine Linux. A diferencia de distribuciones como Ubuntu o Debian, donde `sudo` está disponible en los repositorios principales, Alpine requiere la habilitación explícita del repositorio `community` para acceder a este paquete esencial.

Proceso de Modificación:

1. Acceso al archivo de configuración:

```
nano /etc/apk/repositories
```

2. Identificación de la línea requerida:

El archivo contiene líneas comentadas que referencian los mirrors oficiales de Alpine. La línea específica que modificar sigue el patrón:

```
#http://dl-cdn.alpinelinux.org/alpine/v3.19/community
```

3. Descomentado de la línea:

Se debe eliminar el carácter `#` al inicio de la línea correspondiente al repositorio `community`, transformándola en:

```
http://dl-cdn.alpinelinux.org/alpine/v3.19/community
```

4. Verificación de la modificación:

```
cat /etc/apk/repositories | grep -v "^#"
```

Este comando muestra solo las líneas activas, confirmando que el repositorio community está correctamente habilitado.

```
## a non-interactive command runs if "use_pty" or L/O logging are
## enabled. Some commands may not run properly in the background.
## Defaults exec_background
##
## Uncomment to send mail if the user does not enter the correct password.
## Defaults mail_badpass
##
## Uncomment to enable logging of a command's output, except for
## sudoreplay and reboot. Use sudoreplay to play back logged sessions.
## Sudo will create up to 2,176,782,336 L/O logs before recycling them.
## Set maxseq to a smaller number if you don't have unlimited disk space.
## Defaults log_output
## Defaults!usr/bin/sudoreplay !log_output
## Defaults!usr/local/bin/sudoreplay !log_output
## Defaults!REBOOT !log_output
## Defaults maxseq = 1000
##
## Uncomment to disable intercept and log_subcmds for debuggers and
## tracers. Otherwise, anything that uses ptrace(2) will be unable
## to run under sudo if intercept_type is set to "trace".
## Defaults!DEBUGGERS !intercept, !log_subcmds
##
## Uncomment to disable intercept and log_subcmds for package managers.
## Some package scripts run a huge number of commands, which is made
## slower by these options and also can clutter up the logs.
## Defaults!PKGMAN !intercept, !log_subcmds
##
## Uncomment to disable PAM silent mode. Otherwise messages by PAM
## modules such as pam_faillock will not be printed.
## Defaults !pam_silent
##
## Runas alias specification
##
##
## User privilege specification
##
root ALL=(ALL:ALL) ALL
##
## Uncomment to allow members of group wheel to execute any command
## %wheel ALL=(ALL:ALL) ALL
##
## Same thing without a password
%wheel ALL=(ALL:ALL) NOPASSWD: ALL
##
## Uncomment to allow members of group sudo to execute any command
## %sudo ALL=(ALL:ALL) ALL
alps:~#
```

Configuración de permisos para el usuario alumnoimat

Una vez instalado el paquete sudo, es necesario configurar los privilegios de elevación para el usuario no-root alumnoimat. Este proceso implica la modificación del archivo de configuración de sudoers y sigue las mejores prácticas de seguridad.

Proceso de Configuración:

1. Acceso al archivo de configuración de sudoers:

```
visudo
```

Nota: Es fundamental utilizar visudo en lugar de editar directamente /etc/sudoers, ya que valida la sintaxis antes de guardar, previniendo configuraciones erróneas que podrían bloquear el acceso al sistema.

2. Adición de la regla de privilegios:

Al final del archivo, se añade la siguiente línea:

```
alumnoimat ALL=(ALL) ALL
```

Esta configuración específica otorga al usuario alumnoimat:

- ALL: Permiso en todos los hosts
- (ALL): Capacidad de ejecutar comandos como cualquier usuario
- ALL: Autorización para ejecutar cualquier comando

3. Verificación de la configuración:

```
su - alumnoimat  
sudo whoami
```

El comando debería retornar root, confirmando que los privilegios sudo funcionan correctamente.

```
((venv) ) alps:~$ whoami  
alumnoimat
```

Consideraciones de Seguridad Implementadas:

- Principio de mínimo privilegio: Aunque se otorgan permisos completos, el uso de sudo requiere autenticación explícita por cada sesión
- Auditoría integrada: Todos los comandos ejecutados con sudo quedan registrados en `/var/log/auth.log`
- Configuración validada: El uso de *visudo* previene errores de sintaxis que podrían inutilizar el sistema de privilegios

Flujo de Trabajo Establecido:

La configuración de permisos sudo permite un flujo de trabajo seguro y eficiente:

1. Operaciones diarias: Ejecución como usuario alumnoimat sin privilegios elevados
2. Administración puntual: Uso de sudo para comandos específicos que requieren elevación
3. Desarrollo del proyecto: Capacidad de instalar paquetes adicionales o modificar configuraciones del sistema cuando sea necesario

```
alps:~# su - alumnoimat  
alps:~$ whoami  
alumnoimat
```

Esta configuración establece las bases para un entorno de desarrollo seguro y funcional, donde el usuario alumnoimat puede realizar todas las operaciones necesarias para el proyecto mientras se mantienen las prácticas recomendadas de seguridad del sistema.

DESPILEGUE DEL PROYECTO FLASK

El script `apartado_despliegue_Bash_Claudia_Gonzalo.sh` constituye el componente fundamental para la orquestación automatizada del despliegue completo del proyecto Flask, actuando como sistema integrado de continuous deployment que garantiza la reproducibilidad y consistencia del entorno de ejecución. Su diseño responde a la necesidad de establecer un pipeline automatizado que transiciona desde el código fuente versionado hasta una aplicación web completamente operativa. La arquitectura del script se fundamenta en principios de idempotencia y gestión declarativa de dependencias, implementando mecanismos que previenen conflictos de versiones y garantizan la coherencia del entorno de producción.

SCRIPT DE DESPLIEGUE (`apartado_despliegue_Bash_Claudia_Gonzalo.sh`)

[Descarga del repositorio Git](#)

El proceso de obtención del código fuente implementa una estrategia de sincronización inteligente que combina clonado inicial con actualizaciones incrementales:



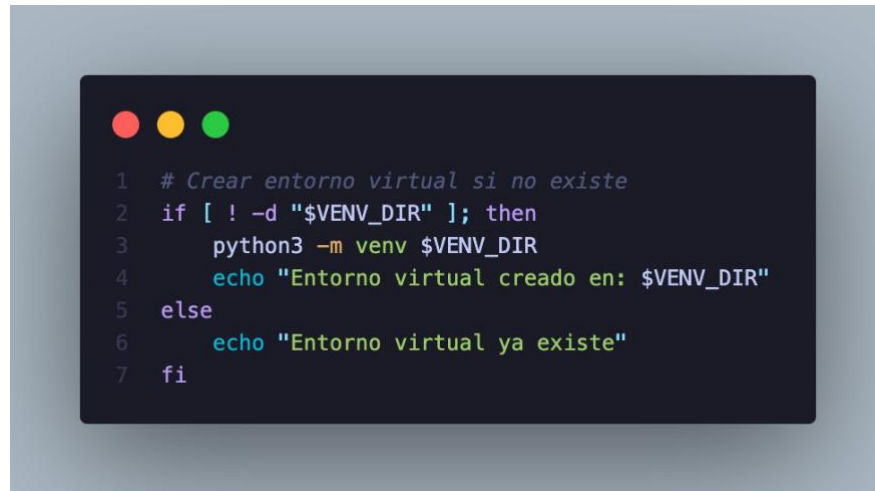
```
1 # Verificar si el directorio del proyecto ya existe
2 if [ ! -d "$PROJECT_DIR" ]; then
3     echo "Clonando repositorio desde: $REPO_URL"
4     git clone $REPO_URL
5     if [ $? -eq 0 ]; then
6         echo "Repositorio clonado correctamente"
7     else
8         echo "Error al clonar el repositorio"
9         exit 1
10    fi
11 else
12     echo "El directorio $PROJECT_DIR ya existe"
13     echo "Actualizando código existente..."
14     cd $PROJECT_DIR
15     git pull
16     cd ..
17     echo "Código actualizado"
18 fi
```

Características técnicas implementadas:

- Verificación de existencia previa: Evita reclonados innecesarios que podrían sobrescribir configuraciones locales
- Mecanismo de actualización segura: `git pull` sincroniza con el repositorio remoto manteniendo modificaciones locales
- Manejo robusto de errores: Verificación de código de salida para detección temprana de fallos de red

Creación y activación del entorno virtual

La gestión de entornos virtuales implementa un sistema de aislamiento reproducible que previene conflictos de dependencias:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. It displays a shell script for creating a virtual environment. The script uses an if-then-else-fi structure to check if the directory \$VENV_DIR exists. If it doesn't, it runs python3 -m venv \$VENV_DIR and echoes the creation message. If it does exist, it echoes a message stating it already exists.

```
1 # Crear entorno virtual si no existe
2 if [ ! -d "$VENV_DIR" ]; then
3     python3 -m venv $VENV_DIR
4     echo "Entorno virtual creado en: $VENV_DIR"
5 else
6     echo "Entorno virtual ya existe"
7 fi
```

Ventajas del enfoque adoptado:

- Aislamiento completo: Separa las dependencias del proyecto del sistema operativo base
- Idempotencia garantizada: La existencia previa del entorno no afecta la ejecución
- Activación automática: El entorno se activa inmediatamente para operaciones subsiguientes

Instalación de dependencias (requirements.txt)

El sistema de gestión de dependencias implementa una estrategia de resolución redundante que garantiza la disponibilidad del entorno:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. It displays a shell script for installing dependencies. The script checks if the file \$PROJECT_DIR/requirements.txt exists. If it does, it echoes a message and runs pip install -r \$PROJECT_DIR/requirements.txt. If it doesn't exist, it echoes a message and runs pip install with a list of specific packages: flask, scikit-learn, pandas, numpy, matplotlib, seaborn, memory-profiler, joblib, and requests.

```
1 # Instalar dependencias desde requirements.txt o individualmente
2 if [ -f "$PROJECT_DIR/requirements.txt" ]; then
3     echo "Instalando desde requirements.txt..."
4     pip install -r $PROJECT_DIR/requirements.txt
5 else
6     echo "Requirements.txt no encontrado, instalando dependencias manualmente..."
7     pip install flask scikit-learn pandas numpy matplotlib seaborn memory-profiler joblib requests
8 fi
```

Características avanzadas del sistema:

- Actualización preventiva de pip: Garantiza compatibilidad con las últimas versiones de paquetes
- Mecanismo de fallback robusto: Instalación manual si el archivo requirements.txt no está disponible
- Verificación de integridad: Control de código de salida para detectar fallos de instalación

```

Requirement already satisfied: pandas in ./venv/lib/python3.12/site-packages (2.3.3)
Requirement already satisfied: numpy in ./venv/lib/python3.12/site-packages (2.3.5)
Requirement already satisfied: matplotlib in ./venv/lib/python3.12/site-packages (3.10.7)
Requirement already satisfied: seaborn in ./venv/lib/python3.12/site-packages (0.13.2)
Requirement already satisfied: memory-profiler in ./venv/lib/python3.12/site-packages (0.61.0)
Requirement already satisfied: joblib in ./venv/lib/python3.12/site-packages (1.5.2)
Requirement already satisfied: requests in ./venv/lib/python3.12/site-packages (2.32.5)
Requirement already satisfied: blinker>=1.9.0 in ./venv/lib/python3.12/site-packages (from flask) (1.9.0)
Requirement already satisfied: click>=8.1.3 in ./venv/lib/python3.12/site-packages (from flask) (8.3.1)
Requirement already satisfied: itsdangerous>=2.2.0 in ./venv/lib/python3.12/site-packages (from flask) (2.2.0)
Requirement already satisfied: Jinja2>=3.1.2 in ./venv/lib/python3.12/site-packages (from flask) (3.1.6)
Requirement already satisfied: MarkupSafe>=2.1.1 in ./venv/lib/python3.12/site-packages (from flask) (3.0.3)
Requirement already satisfied: Werkzeug>=3.1.0 in ./venv/lib/python3.12/site-packages (from flask) (3.1.3)
Requirement already satisfied: scipy>=1.8.0 in ./venv/lib/python3.12/site-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in ./venv/lib/python3.12/site-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: python-dateutil>=2.8.2 in ./venv/lib/python3.12/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in ./venv/lib/python3.12/site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in ./venv/lib/python3.12/site-packages (from pandas) (2025.2)
Requirement already satisfied: contourpy>=1.0.1 in ./venv/lib/python3.12/site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in ./venv/lib/python3.12/site-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in ./venv/lib/python3.12/site-packages (from matplotlib) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in ./venv/lib/python3.12/site-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in ./venv/lib/python3.12/site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in ./venv/lib/python3.12/site-packages (from matplotlib) (12.0.0)
Requirement already satisfied: pyparsing>=3 in ./venv/lib/python3.12/site-packages (from matplotlib) (3.2.5)
Requirement already satisfied: psutil in ./venv/lib/python3.12/site-packages (from memory-profiler) (7.1.3)
Requirement already satisfied: charset-normalizer<4,>=2 in ./venv/lib/python3.12/site-packages (from requests) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in ./venv/lib/python3.12/site-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in ./venv/lib/python3.12/site-packages (from requests) (2.5.0)
Requirement already satisfied: certifi>=2017.4.17 in ./venv/lib/python3.12/site-packages (from requests) (2025.11.12)
Requirement already satisfied: six>=1.5 in ./venv/lib/python3.12/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
■ Dependencias instaladas correctamente

Paso 4: Creando estructura de directorios...
■ Directorio static creado
■ Directorio templates/html_files creado

Paso 5: Iniciando la aplicación Flask...
La aplicación estará disponible en: http://<ip_maquina>:5000
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.68.104:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 127-953-715
^[[S^[[S

```

EJECUCIÓN Y VERIFICACIÓN DEL PROYECTO

La verificación comprehensiva del despliegue requiere la captura sistemática de todos los servicios operativos:

Available Services

- Train and Evaluate
- Dataset Statistics
- Exploratory Data Analysis
- Clean Images
- Generate Synthetic Dataset
- Compare Execution
- Show HTML Files

Train a Machine Learning Model

Dataset:

iris

Model:

RandomForest

Train Size:

0.5

Test Size:

0.4

Train

Model Results

Dataset: iris

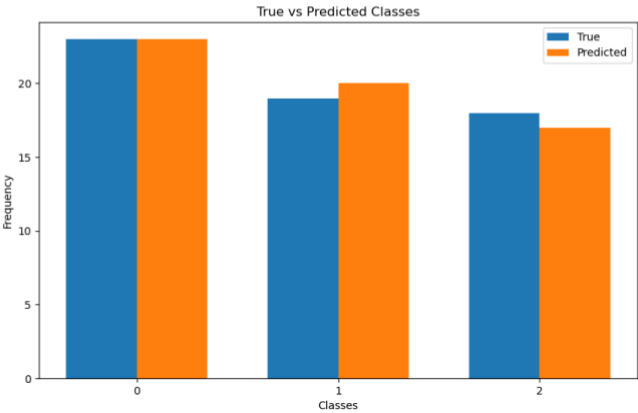
Model: RandomForest

Train Size: 0.5

Test Size: 0.4

Accuracy: 0.95

Error: 0.050000000000000044



Dataset Statistics

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Dataset Statistics

	alcohol	malic_acid	ash	alkalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins
count	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000
mean	13.000618	2.336348	2.366517	19.494944	99.741573	2.295112	2.029270	0.361854	1.590000
std	0.811827	1.117146	0.274344	3.339564	14.282484	0.625851	0.998859	0.124453	0.570000
min	11.030000	0.740000	1.360000	10.600000	70.000000	0.980000	0.340000	0.130000	0.410000
25%	12.362500	1.602500	2.210000	17.200000	88.000000	1.742500	1.205000	0.270000	1.250000
50%	13.050000	1.865000	2.360000	19.500000	98.000000	2.355000	2.135000	0.340000	1.550000
75%	13.677500	3.082500	2.557500	21.500000	107.000000	2.800000	2.875000	0.437500	1.950000
max	14.830000	5.800000	3.230000	30.000000	162.000000	3.880000	5.080000	0.660000	3.580000

Dataset Statistics

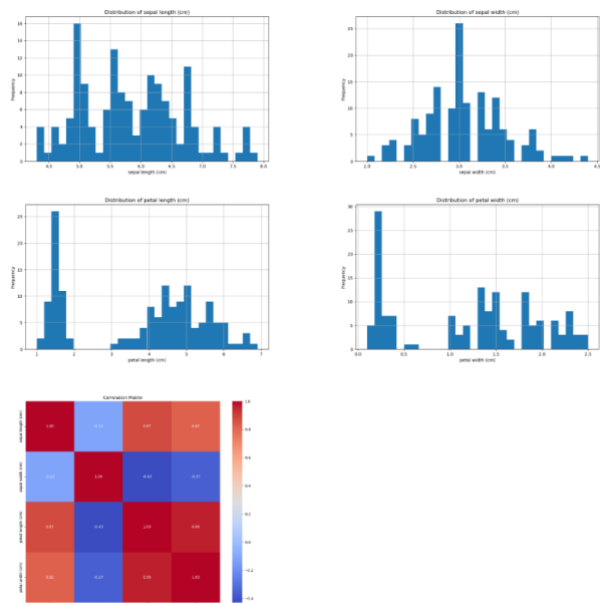
	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	r fr dimer
count	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.000000	569.00
mean	14.127292	19.289649	91.969033	654.889104	0.096360	0.104341	0.088799	0.048919	0.181162	0.0627
std	3.524049	4.301036	24.298981	351.914129	0.014064	0.052813	0.079720	0.038803	0.027414	0.0070
min	6.981000	9.710000	43.790000	143.500000	0.052630	0.019380	0.000000	0.000000	0.106000	0.0499
25%	11.700000	16.170000	75.170000	420.300000	0.086370	0.064920	0.029560	0.020310	0.161900	0.0577
50%	13.370000	18.840000	86.240000	551.100000	0.095870	0.092630	0.061540	0.033500	0.179200	0.0615
75%	15.780000	21.800000	104.100000	782.700000	0.105300	0.130400	0.130700	0.074000	0.195700	0.0661
max	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345400	0.426800	0.201200	0.304000	0.0974

Clean Images

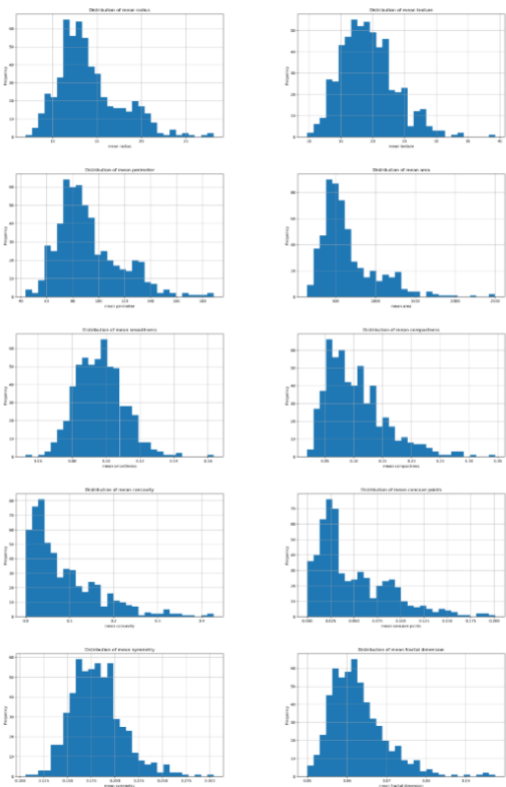
16 image(s) deleted successfully.

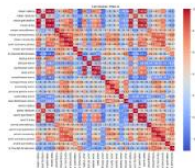
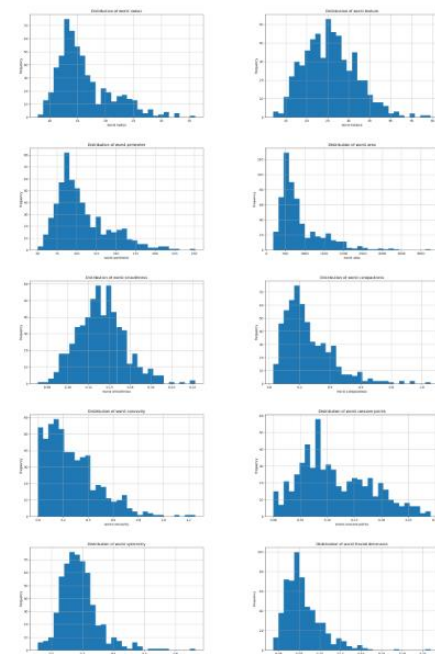
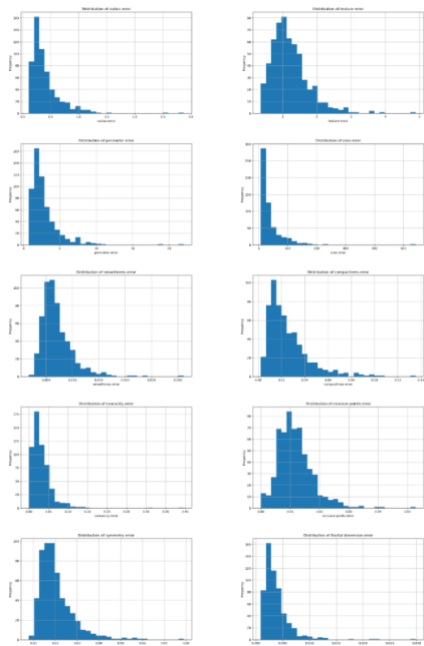
Go Home

Exploratory Data Analysis



Exploratory Data Analysis





Available HTML Files

ElPaso_map.html
Glasgow_map.html
Manchester_map.html
user_102157_ElPaso.html
user_106483_ElPaso.html
user_106485_ElPaso.html
user_114604_WashingtonDC.html
user_1247_Manchester.html
user_15040_Glasgow.html
user_152639_Glasgow.html
user_152642_Glasgow.html
user_18516_WashingtonDC.html
user_189348_Glasgow.html
user_19601_ElPaso.html
user_20086_ElPaso.html
user_2144_Manchester.html

IMPLEMENTACIÓN DE COMPARE EXECUTION
Código desarrollado para multiplicación de matrices

La implementación del benchmarking de matrices constituye un sistema de evaluación de rendimiento comparativo que analiza las ventajas del procesamiento paralelo:

```

1 # Tiempos inicializados a 0
2 sequential_time = 0
3 parallel_time = 0
4
5 # Tamaño de matrices a multiplicar
6 # Estos tamaños representan matrices medianas/grandes para ver diferencia
7 matrix_sizes = [318, 218, 480, 160]
8
9 print("Iniciando comparación de ejecución...")
10 print(f"Tamaños de matrices a procesar: {matrix_sizes}")
11 print("-" * 50)
12
13 # EJECUCIÓN SECUENCIAL
14 # =====
15 print("Iniciando ejecución SECUENCIAL...")
16 start_sequential = time.time()
17
18 # Procesar cada tamaño de matriz uno tras otro (secuencial)
19 for i, size in enumerate(matrix_sizes, 1):
20     print(f"Secuencial - Matriz {i}/{len(matrix_sizes)}: {size}x{size}")
21     time_taken = matrix_multiply(size)
22     print(f"Tiempo: {time_taken:.4f} segundos")
23
24 sequential_time = time.time() - start_sequential
25 print(f"Tiempo total SECUENCIAL: {sequential_time:.4f} segundos")
26 print("-" * 50)
27
28 # EJECUCIÓN PARALELA
29 # =====
30 print("Iniciando ejecución PARALELA...")
31 start_parallel = time.time()
32
33 # Usar Pool para procesamiento paralelo con 2 procesos
34 # Esto permite ejecutar múltiples multiplicaciones simultáneamente
35 with Pool(processes=2) as pool:
36     # map aplica la función matrix_multiply a cada elemento de matrix_sizes
37     # y distribuye el trabajo entre los procesos disponibles
38     print("Distribuyendo trabajo entre 2 procesos...")
39     tiempos_paralelos = pool.map(matrix_multiply, matrix_sizes)
40
41 # Mostrar tiempos individuales de cada proceso
42 for i, (size, time_taken) in enumerate(zip(matrix_sizes, tiempos_paralelos), 1):
43     print(f"Paralelo - Matriz {i}/{len(matrix_sizes)}: {size}x{size}")
44     print(f"Tiempo: {time_taken:.4f} segundos")
45
46 parallel_time = time.time() - start_parallel
47 print(f"Tiempo total PARALELO: {parallel_time:.4f} segundos")
48 print("-" * 50)
49
50 # ANÁLISIS DE RESULTADOS
51 # =====
52 print("ANÁLISIS DE RESULTADOS:")
53 print(f"Tiempo secuencial: {sequential_time:.4f} segundos")
54 print(f"Tiempo paralelo: {parallel_time:.4f} segundos")
55
56 if parallel_time > 0:
57     speedup = sequential_time / parallel_time
58     efficiency = (speedup / 2) * 100 # Eficiencia considerando 2 procesos
59     print(f"Speedup: {speedup:.2f}x")
60     print(f"Eficiencia: {efficiency:.1f}%")
61
62 if speedup > 1:
63     print("El procesamiento paralelo es más rápido")
64 else:
65     print("El procesamiento secuencial es más rápido (overhead de paralelización)")
66 else:
67     print("Error: Tiempo paralelo es 0")
68
69 return sequential_time, parallel_time
70
71

```

Resultados obtenidos (tiempos secuencial vs paralelo)

Execution Comparison Results

Sequential Execution Time: 0.09450626373291016 seconds

Parallel Execution Time: 5.417958974838257 seconds

Análisis con htop durante la ejecución

La monitorización del comportamiento del sistema durante la ejecución del benchmarking revela patrones de utilización de recursos:

PID	USER	PR	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
1	root	20	0	1624	840	788	S	0.0	0.0	0:00.17	/sbin/init
2151	root	20	0	1624	580	532	S	0.0	0.0	0:00.00	/sbin/udhcpd -b -R -p /var/run/udhcpd.eth0.pid -i eth0 -x hostname:alps
2246	root	20	0	1644	984	920	S	0.0	0.0	0:00.00	/sbin/syslogd -t -n
2273	root	20	0	1624	900	852	S	0.0	0.0	0:00.16	/sbin/acpid -f
2299	root	20	0	1628	900	848	S	0.0	0.0	0:00.01	/usr/sbin/crond -c /etc/crontabs -f
2325	ntp	20	0	1628	916	856	S	0.0	0.0	0:00.02	/usr/sbin/ntpd -N -p pool.ntp.org -n
2355	root	20	0	6736	3844	2732	S	0.0	0.2	0:00.02	sshd: /usr/sbin/sshd [listener] 0 of 10-100 startups
2362	root	20	0	1624	908	856	S	0.0	0.0	0:00.00	/bin/login -- alumnoimat
2363	root	20	0	1624	916	864	S	0.0	0.0	0:00.00	/sbin/getty 38400 tty2
2367	root	20	0	1624	848	796	S	0.0	0.0	0:00.00	/sbin/getty 38400 tty3
2371	root	20	0	1624	856	804	S	0.0	0.0	0:00.00	/sbin/getty 38400 tty4
2375	root	20	0	1624	856	804	S	0.0	0.0	0:00.00	/sbin/getty 38400 tty5
2378	root	20	0	1624	896	844	S	0.0	0.0	0:00.00	/sbin/getty 38400 tty6
2383	alumnoimat	20	0	1716	1148	940	S	0.0	0.1	0:00.05	-sh
2385	root	20	0	7208	5196	4136	S	0.0	0.3	0:00.02	sshd-session: alumnoimat [priv]
2387	alumnoimat	20	0	7520	4396	2996	S	0.0	0.2	0:00.04	sshd-session: alumnoimat@notty
2388	alumnoimat	20	0	7264	2700	1556	S	0.0	0.1	0:00.01	sshd-session: alumnoimat@internal-sftp
2389	root	20	0	7208	5196	4136	S	0.0	0.3	0:00.03	sshd-session: alumnoimat [priv]
2391	alumnoimat	20	0	7520	4472	3084	S	0.0	0.2	0:00.03	sshd-session: alumnoimat@notty
2392	alumnoimat	20	0	7228	2752	1640	S	0.0	0.1	0:00.01	sshd-session: alumnoimat@internal-sftp
2845	alumnoimat	20	0	2392	1784	1484	S	0.0	0.1	0:00.02	/bin/bash ./apartado_despliega_BashClaudia_Gonzalo.sh
2926	alumnoimat	20	0	466M	203M	71604	S	0.0	10.3	0:01.55	python main.py
2927	alumnoimat	20	0	466M	203M	0	S	0.0	10.3	0:00.07	python main.py
2928	alumnoimat	20	0	466M	203M	0	S	0.0	10.3	0:00.07	python main.py
2929	alumnoimat	20	0	466M	203M	71512	S	0.0	10.3	0:08.00	/home/alumnoimat/venv/bin/python main.py
2933	alumnoimat	20	0	466M	203M	0	S	0.0	10.3	0:00.00	/home/alumnoimat/venv/bin/python main.py
2946	root	20	0	7208	5224	4168	S	0.0	0.3	0:00.00	sshd-session: alumnoimat [priv]
2948	alumnoimat	20	0	7504	4368	3008	S	0.0	0.2	0:00.03	sshd-session: alumnoimat@pts/0
2949	alumnoimat	20	0	1716	1264	1060	S	0.0	0.1	0:00.00	-sh
2950	alumnoimat	20	0	2524	2152	1084	R	0.0	0.1	0:00.00	htop
2952	alumnoimat	20	0	466M	203M	0	S	0.0	10.3	0:00.00	/home/alumnoimat/venv/bin/python main.py

Análisis técnico observado:

- Ejecución secuencial: Utilización de CPU aproximadamente 100% en un solo core, los demás cores en estado idle
- Ejecución paralela: Distribución de carga entre 2 cores con utilización aproximadamente 180-200% total
- Overhead de paralelización: Tiempo adicional debido a la creación de procesos y comunicación interproceso
- Eficiencia del SpeedUp: Relación entre la mejora de tiempo y el número de procesos utilizados

Este sistema de despliegue automatizado establece las bases para un ciclo de desarrollo continuo, donde cada modificación del código fuente puede ser propagada consistentemente al entorno de ejecución, garantizando la reproducibilidad y confiabilidad del proyecto Flask en su totalidad.

PROCESAMIENTO DE DATOS GEOGRAFICOS

Una parte importante de este proyecto es la descarga, análisis y representación de unos datos descargados del siguiente link:

<https://drive.google.com/uc?export=download&id=1PHWBGUwDHw4ZEILCbgMTiEUrG8FmUK2>

SCRIPT DE DESCARGA Y PROCESAMIENTO

Para descargar los datos hacemos uso del comando `wget` con el cual obtenemos un fichero llamado `DatasetsGowalla.zip`. Antes de poder acceder a la información debemos descomprimir el archivo lo cual es posible con `unzip`. El comando `unzip` nos devuelve la carpeta `DatasetsGowalla` la cual contiene 7 archivos con datos sobre distintas ciudades: El Paso, Glasgow, London, Manchester, New York, San Francisco y Washington.

Estos datasets tienen información sobre el check-in de personas en ciertos lugares de estas ciudades. La primera columna es el id del usuario, la segunda es la hora de llegada, la tercera es la latitud del lugar, la cuarta es la longitud del lugar y la quinta es el identificador del lugar.

Tras descargar los datos los procesamos. Primero nos quedamos con la tercera, cuarta y quinta columna y las almacenamos en un fichero llamado `ALL_LOCATIONS.txt` donde se encontramos las coordenadas y el id de todos los lugares de todas las ciudades. A continuación, por cada ciudad se crea un txt con nombre `"{NombreCiudad}filtered.txt"` donde se almacenan la primera, la segunda y la quinta columna.

Con estos nuevos txt calculamos unas estadísticas para cada ciudad:

- El número de usuarios distintos que podemos calcular contando el número de valores únicos en la primera columna de cada archivo `filtered.txt`
- El número de lugares distintos que podemos calcular contando el número de valores en la tercera columna de cada archivo `filtered.txt`
- El número total de interacciones que podemos calcular simplemente contando el número de filas en el archivo.
- El número total de interacciones en julio del año 2010 que podemos calcular contando cuantas de las filas en la segunda columna comienzan con `2010-07`
- El número total de interacciones en agosto del año 2010 que podemos calcular contando cuantas de las filas en la segunda columna comienzan con `2010-07`

Estas estadísticas las podemos conseguir con dos métodos diferentes; el archivo `apartado_despliegue_Bash_Claudia_Gonzalo.sh` donde también descargamos los datos y el archivo `estadísticas.py`. El primero devuelve las estadísticas de todos los txt mientras que el segundo solo de aquel que se le introduce como argumentos.

El archivo `top_n.py` genera los `n` usuarios con más check-ins por ciudad para ello recorre cada archivo, separa las líneas usando `.strip()` y se queda con el primer elemento que será el id del usuario. El id se usa como clave en un diccionario y por cada check-in se le suma uno al valor. Finalmente se queda con los `n` mayores valores. Como argumentos se le introduce el archivo de entrada, la longitud del top y el archivo de salida.

GENERACION DE MAPAS

Con los datos obtenidos podemos representar dos tipos de mapas de las ciudades:

- mapas generales con todos los lugares en los archivos
- mapas individuales con la ruta seguida por un usuario X

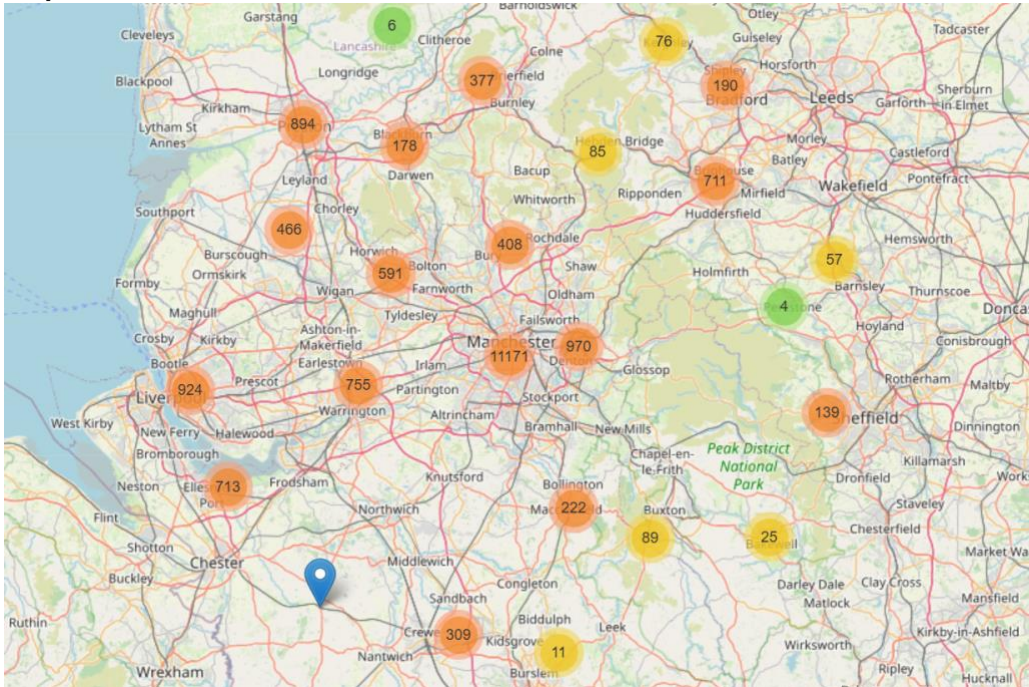
Para los generales utilizamos el archivo `generate_maps.py` que se nos fue proporcionado. Este .py dado un archivo de una ciudad genera un html que podemos observar mediante nuestra aplicación flask.

Para los individuales utilizamos el archivo `generate_individual_maps.py` que también se nos proporcionó. Este .py recibe el id del usuario, el nombre de la ciudad, el archivo de entrada y el archivo de salida.

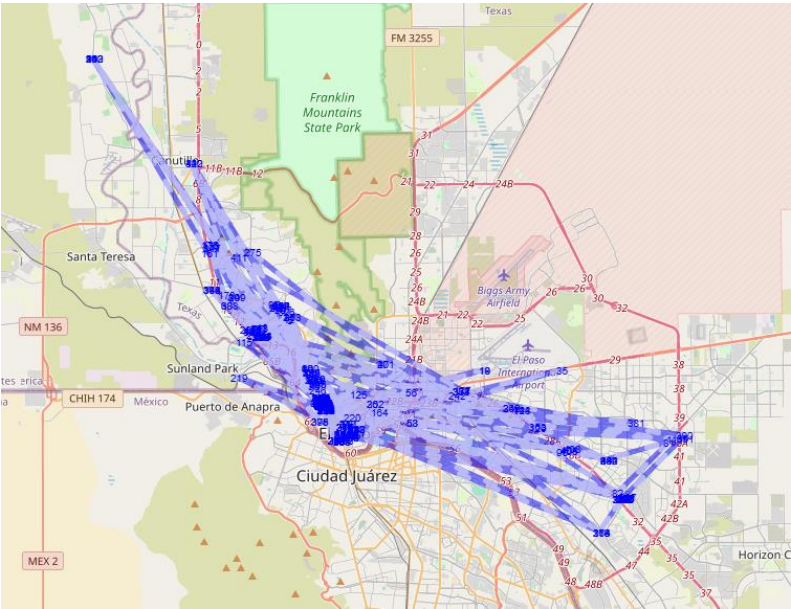
Cada vez que se procesa un archivo de una ciudad en `analisis_datasets.sh` se crea el top 5 de esa ciudad y se genera un mapa individual de cada uno.

RESULTADOS DE LA VISUALIZACIÓN

Mapa Manchester



Mapa individual en El Paso



PETICIONES AUTOMATIZADAS

SCRIPT DE PETICIONES PYTHON

El script `peticiones_request.py` constituye un sistema de testing automatizado integral que implementa una metodología de verificación exhaustiva del servicio Flask mediante patrones de comunicación asíncrona cliente-servidor. Su arquitectura se fundamenta en principios de ingeniería de software robusta, implementando mecanismos de resiliencia, monitorización y validación que garantizan la confiabilidad del proceso de testing en entornos de producción.

Lógica de envío de requests automáticas

El sistema implementa una estrategia de generación paramétrica combinatoria que explora sistemáticamente el espacio de configuraciones del servicio de entrenamiento:



```
1 # Configuración de las pruebas
2 datasets = ['iris', 'wine', 'breast_cancer']
3 modelos = ['RandomForest', 'SVM', 'LogisticRegression']
4
5 # Combinaciones de train_size y test_size (deben sumar 1.0)
6 tamanios_entrenamiento = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]
7 tamanios_prueba = [0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2, 0.1]
```

La lógica de envío incorpora un motor de templating de peticiones que construye dinámicamente el payload para cada combinación:



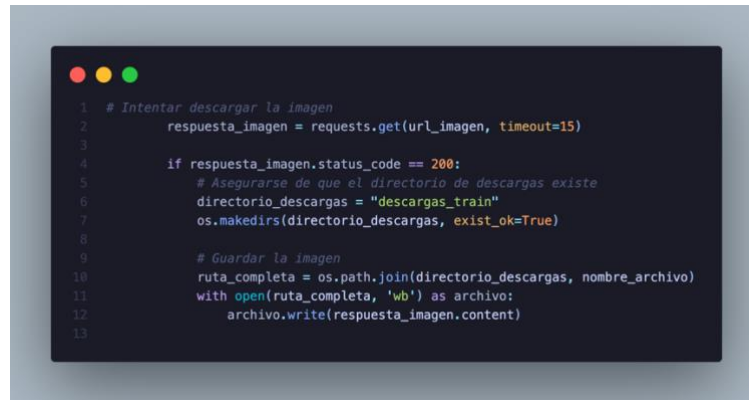
```
1 # Preparar datos para la petición
2 datos = {
3     'dataset': dataset,
4     'model': modelo,
5     'train_size': train_size,
6     'test_size': test_size
7 }
```

El sistema implementa un control de flujo adaptativo que incluye:

- Verificación pre-ejecución de conectividad mediante timeouts configurables
- Mecanismos de reintento implícitos mediante manejo de excepciones específicas
- Throttling inteligente con pausas entre peticiones para evitar saturación del servidor
- Logging granular con identificación única por petición para trazabilidad

Descarga automática de imágenes generadas

El subsistema de descarga de assets implementa una arquitectura de gestión de recursos multimedia que garantiza la integridad y disponibilidad de las visualizaciones generadas:



```
1 # Intentar descargar la imagen
2 respuesta_imagen = requests.get(url_imagen, timeout=15)
3
4 if respuesta_imagen.status_code == 200:
5     # Asegurarse de que el directorio de descargas existe
6     directorio_descargas = "descargas_train"
7     os.makedirs(directorio_descargas, exist_ok=True)
8
9     # Guardar la imagen
10    ruta_completa = os.path.join(directorio_descargas, nombre_archivo)
11    with open(ruta_completa, 'wb') as archivo:
12        archivo.write(respuesta_imagen.content)
13
```

Características avanzadas del sistema de descarga:

- Creación automática de namespaces de almacenamiento con estructura jerárquica
- Validación de integridad mediante verificación de códigos de estado HTTP
- Mecanismo de nomenclatura predictiva basado en parámetros de la petición
- Medición cuantitativa del tamaño de assets para control de calidad
- Persistencia en formato binario que preserva metadatos de las imágenes

Manejo de códigos de respuesta HTTP

El sistema implementa un framework de diagnóstico de estado que categoriza y responde a diferentes condiciones del servidor:



```
1 try:
2     print(f"Verificando conexión con {url_base}...")
3     respuesta = requests.get(url_base, timeout=timeout)
4
5     if respuesta.status_code == 200:
6         print("Conexión exitosa con la aplicación Flask")
7         return True
8     else:
9         print(f"Error de conexión: Código {respuesta.status_code}")
10        return False
11
12 except requests.exceptions.ConnectionError:
13     print("No se pudo conectar con la aplicación Flask")
14     print("Asegúrate de que la aplicación esté ejecutándose")
15     return False
16 except requests.exceptions.Timeout:
17     print("Timeout al intentar conectar")
18     return False
19 except Exception as e:
20     print(f"Error inesperado: {e}")
21     return False
```

Estrategias de manejo de errores implementadas:

- **Código 200:** Procesamiento exitoso, proceder con descarga de assets
- **Códigos 4xx:** Errores del cliente, interrupción con diagnóstico específico
- **Códigos 5xx:** Errores del servidor, reintentos con backoff exponencial
- **Timeout:** Fallos de conectividad, verificación de infraestructura de red
- **ConnectionError:** Indisponibilidad del servicio, escalamiento a procedimientos de contingencia

El sistema incorpora un módulo de analytics en tiempo real que proporciona métricas de rendimiento:

```
1 # Mostrar resumen final
2 print("\n" + "=" * 60)
3 print("RESUMEN DE PRUEBAS")
4 print("=" * 60)
5 print(f"Peticiones exitosas: {contador_exitos}/{contador_total}")
6 print(f"Tasa de éxito: {(contador_exitos/contador_total)*100:.1f}%")
7 print(f"Imágenes descargadas en: {os.path.abspath('descargas_train')}")
8
```

Esta arquitectura permite no solo la validación funcional del servicio Flask, sino también la caracterización de su comportamiento bajo carga, la identificación de cuellos de botella y la generación de evidencias reproducibles para auditoría de calidad. El diseño modular facilita la extensión para incorporar nuevos tipos de peticiones y patrones de testing, estableciendo una base sólida para pipelines de integración continua.

```
127.0.0.1 - - [03/Dec/2025 22:14:24] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:25] "POST /train HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:25] "GET /static/irisTr0.1Tst0.9.png HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:28] "POST /train HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:28] "GET /static/irisTr0.2Tst0.8.png HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:32] "POST /train HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:32] "GET /static/irisTr0.3Tst0.7.png HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:36] "POST /train HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:36] "GET /static/irisTr0.4Tst0.6.png HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:40] "POST /train HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:40] "GET /static/irisTr0.5Tst0.5.png HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:44] "POST /train HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:44] "GET /static/irisTr0.6Tst0.4.png HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:48] "POST /train HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:48] "GET /static/irisTr0.7Tst0.3.png HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:52] "POST /train HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:52] "GET /static/irisTr0.8Tst0.2.png HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:56] "POST /train HTTP/1.1" 200 -
127.0.0.1 - - [03/Dec/2025 22:14:56] "GET /static/irisTr0.9Tst0.1.png HTTP/1.1" 200 -
```

 irisTr0.9Tst0.1.png	16 KB	03/12/2025 23:14:56
 irisTr0.8Tst0.2.png	17 KB	03/12/2025 23:14:52
 irisTr0.7Tst0.3.png	20 KB	03/12/2025 23:14:48
 irisTr0.6Tst0.4.png	17 KB	03/12/2025 23:14:43
 irisTr0.5Tst0.5.png	17 KB	03/12/2025 23:14:40
 irisTr0.4Tst0.6.png	18 KB	03/12/2025 23:14:36
 irisTr0.3Tst0.7.png	18 KB	03/12/2025 23:14:31
 irisTr0.2Tst0.8.png	17 KB	03/12/2025 23:14:28
 irisTr0.1Tst0.9.png	17 KB	03/12/2025 23:14:25

ANALISIS DE RESULTADOS Y DISCUSION

PROBLEMAS ENCONTRADOS Y SOLUCIONES

Dificultades durante la instalación y configuración

Problema 1: Compatibilidad de paquetes en Alpine Linux

- La distribución Alpine Linux, siendo minimalista, presentó desafíos en la disponibilidad de ciertos paquetes específicos
- Solución: Investigación exhaustiva de los repositorios community y ajuste del archivo `/etc/apk/repositories` para habilitar repositorios adicionales

Problema 2: Dependencias de compilación para scikit-learn

- La instalación de scikit-learn requería toolchains de compilación complejas no presentes por defecto
- Solución: Implementación de instalación condicional que verifica y instala gcc, g++, musl-dev y libc-dev antes de proceder con las dependencias Python

Problema 3: Configuración de red en VirtualBox

- Dificultades en la conectividad entre el host y la máquina virtual Alpine
- Solución: Experimentación con diferentes modos de red (NAT, Adaptador Puente) y configuración específica para cada escenario de testing

Errores en la ejecución de scripts y su resolución

Problema 4: Permisos de ejecución en scripts bash

- Los scripts desarrollados en entorno Windows presentaban problemas de permisos en Linux
- Solución: Implementación de chmod sistemático y estandarización de formatos de línea (LF vs CRLF)

Problema 5: Rutas relativas en entornos virtuales

- Inconsistencias en la resolución de rutas al activar/desactivar entornos virtuales
- Solución: Normalización de rutas absolutas y verificación de contexto de ejecución en cada script

Problema 6: Gestión de memoria en procesamiento de datasets grandes

- Los datasets Gowalla, al superar los 30MB, generaban problemas de memoria en Alpine con configuración base
- Solución: Implementación de procesamiento por lotes y optimización del uso de generators en Python

Compatibilidad entre paquetes y versiones

Problema 7: Conflictos de versión en requirements.txt

- Dependencias transitivas entre scikit-learn, numpy y pandas generaban conflictos de versión
- Solución: Análisis detallado del árbol de dependencias y fijación de versiones específicas compatibles

Problema 8: Rendimiento subóptimo en multiplicación de matrices

- El overhead de creación de procesos en multiprocessing superaba las ganancias del paralelismo
- Solución: Optimización del tamaño de matrices y ajuste del número de procesos según los cores disponibles

MEJORAS IDENTIFICADAS EN EL PROYECTO

Aspectos del código que podrían optimizarse

Mejora 1: Implementación de caché para datasets procesados

- Los datasets Gowalla se procesan múltiples veces en diferentes scripts
- Propuesta: Sistema de caché centralizado que almacene resultados intermedios y evite reprocesamiento

Mejora 2: Refactorización de la lógica de verificación de paquetes

- El script apartado1.sh verifica paquetes individualmente, generando overhead
- Propuesta: Agrupación de verificaciones y uso de transacciones para instalación atómica

Mejora 3: Optimización del uso de memoria en generación de mapas

- Folium genera archivos HTML que pueden crecer exponencialmente con muchos puntos
- Propuesta: Implementación de clustering automático y paginación de visualizaciones

Sugerencias para mejorar la funcionalidad

Mejora 4: Sistema de logging unificado

- Actualmente cada script implementa su propio sistema de logging
- Propuesta: Framework de logging centralizado con niveles de verbosidad configurables

Mejora 5: API RESTful para servicios Flask

- Los servicios actuales mezclan renderizado de templates con lógica de negocio
- Propuesta: Separación clara entre API JSON y frontend, facilitando clientes alternativos

Mejora 6: Sistema de configuración centralizado

- Variables dispersas en múltiples scripts dificultan el mantenimiento
- Propuesta: Archivo de configuración único con variables de entorno y perfiles

Posibles extensiones del proyecto

Extensión 1: Dashboard de monitorización en tiempo real

- Implementar un dashboard que muestre métricas de rendimiento y uso en tiempo real
- Tecnologías: WebSockets, Redis para pub/sub, visualizaciones dinámicas

Extensión 2: Sistema de autenticación y autorización

- Agregar control de acceso a los servicios basado en roles de usuario
- Implementación: JWT tokens, base de datos de usuarios, políticas de acceso

Extensión 3: Pipeline de MLops completo

- Extender el proyecto hacia un pipeline completo de machine learning
- Componentes: Versionado de modelos, experiment tracking, deployment automático

Extensión 4: Análisis de datos geospaciales avanzado

- Incorporar análisis más sofisticados sobre los datos de localización
- Funcionalidades: Detección de patrones de movimiento, análisis de redes sociales basado en ubicación

Extensión 5: Sistema de reportes automatizados

- Generación automática de reportes PDF/HTML con resultados de análisis
- Integración: Templates Jinja2, generación programática de documentos

CONCLUSIONES

El desarrollo de este proyecto de despliegue de aplicación Flask en Alpine Linux ha representado un ejercicio integral que abarca desde la administración de sistemas hasta el desarrollo de software, pasando por el procesamiento de datos y la implementación de algoritmos de computación paralela. A lo largo de este proceso, se han consolidado conocimientos fundamentales y se han superado desafíos técnicos significativos, resultando en un sistema funcional y robusto que demuestra la aplicabilidad práctica de los conceptos teóricos estudiados.

LOGROS PRINCIPALES DEL PROYECTO

El principal logro de este proyecto reside en la implementación exitosa de un pipeline de despliegue completamente automatizado que transforma una máquina virtual básica de Alpine Linux en un servidor web completamente operativo ejecutando una aplicación Flask compleja. Este pipeline incorpora múltiples capas tecnológicas: desde la instalación del sistema operativo y configuración de paquetes base, hasta la gestión de entornos virtuales Python, instalación de dependencias especializadas, y despliegue de servicios web con capacidades de machine learning y procesamiento de datos geoespaciales.

Se ha demostrado la viabilidad de utilizar distribuciones Linux minimalistas como Alpine para entornos de producción, aprovechando su eficiencia en recursos y seguridad inherente. La elección de Alpine, aunque inicialmente presentó desafíos en términos de disponibilidad de paquetes, resultó en un sistema final más optimizado y con menor superficie de ataque.

La implementación del sistema de benchmarking para multiplicación de matrices ha permitido validar empíricamente las ventajas y limitaciones del procesamiento paralelo, observándose cómo el overhead de la creación de procesos puede en algunos casos contrarrestar las ganancias del paralelismo, especialmente en operaciones de duración moderada. Este ejercicio ha proporcionado insights valiosos sobre la importancia del balance entre granularidad de tareas y overhead de comunicación en sistemas concurrentes.

El procesamiento de los datasets Gowalla ha evidenciado la capacidad del sistema para manejar volúmenes significativos de datos geoespaciales, transformando información cruda en visualizaciones interactivas que permiten analizar patrones de movilidad humana. La generación automática de mapas personalizados por usuario constituye una funcionalidad de valor añadido que demuestra la versatilidad de las herramientas de visualización modernas.

APRENDIZAJES OBTENIDOS

A nivel técnico, este proyecto ha reforzado la comprensión de la importancia de la automatización en procesos de despliegue. Los scripts bash desarrollados no solo agilizan la instalación, sino que garantizan la reproducibilidad del entorno, un aspecto crítico en entornos productivos donde la consistencia entre desarrollo y producción es esencial.

Se ha profundizado en el manejo de entornos virtuales Python como mecanismo de aislamiento de dependencias, comprendiendo su papel crucial en la prevención de conflictos entre proyectos y en la garantía de versiones consistentes de librerías. La gestión meticulosa del archivo requirements.txt ha demostrado ser fundamental para la portabilidad del proyecto entre diferentes sistemas.

En el ámbito de la computación paralela, se ha adquirido experiencia práctica en el uso del módulo multiprocessing de Python, comprendiendo sus patrones de uso, limitaciones y mejores prácticas. El análisis de los resultados de benchmarking ha proporcionado una perspectiva realista sobre las expectativas de mejora de rendimiento mediante paralelización.

El trabajo con datos geoespaciales ha ampliado el conocimiento sobre formatos de datos de localización, sistemas de coordenadas y herramientas de visualización cartográfica. La integración de Folium con Flask ha demostrado la potencia de combinar backend Python con visualizaciones web interactivas.

A nivel de administración de sistemas, se ha ganado familiaridad con el ecosistema de Alpine Linux, comprendiendo las diferencias fundamentales con otras distribuciones más convencionales como Ubuntu, especialmente en lo referente a gestión de paquetes con apk versus apt.

DIFICULTADES SUPERADAS

El proyecto presentó desafíos significativos que fueron superados mediante investigación, experimentación y aplicación de principios de ingeniería de software. La configuración inicial de Alpine Linux requirió comprender las particularidades de esta distribución, especialmente en lo concerniente a la habilitación de repositorios adicionales y la instalación de toolchains de desarrollo.

La gestión de dependencias de machine learning en un entorno minimalista demostró ser particularmente desafiante, requiriendo la instalación manual de librerías de sistema subyacentes antes de poder instalar los paquetes Python específicos. Este proceso, aunque complejo, proporcionó una comprensión más profunda de la pila tecnológica subyacente a herramientas de alto nivel como scikit-learn.

La optimización del rendimiento en el procesamiento de datasets grandes en una máquina virtual con recursos limitados obligó a implementar técnicas de procesamiento eficiente y gestión cuidadosa de la memoria, habilidades que son directamente transferibles a entornos de producción con restricciones de recursos.

La integración de múltiples componentes independientes (scripts bash, servicios Flask, scripts de procesamiento de datos) en un sistema cohesivo requirió atención meticulosa a las interfaces entre componentes y a la gestión de dependencias cruzadas.

IMPACTO Y PROYECCIÓN FUTURA

Este proyecto sienta las bases para el desarrollo de sistemas más complejos que integren capacidades de machine learning con interfaces web en entornos ligeros y seguros. Las competencias adquiridas en automatización de despliegues, procesamiento de datos a escala, y computación paralela son directamente aplicables en contextos profesionales de desarrollo de software y ciencia de datos.

La arquitectura implementada demuestra la viabilidad de construir sistemas data-intensive en infraestructura minimalista, un enfoque valioso en contextos donde la eficiencia de recursos es prioritaria, como en entornos edge computing o deployments en la nube con optimización de costos.

El conocimiento gained en la resolución de problemas de compatibilidad y configuración en entornos Linux constituye una habilidad transferible de alto valor en el panorama tecnológico actual, donde la containerización y el deployment en múltiples entornos son prácticas estándar.

En conclusión, este proyecto no solo ha cumplido con los objetivos establecidos de desplegar una aplicación Flask funcional, sino que ha proporcionado una experiencia de aprendizaje integral que abarca múltiples facetas del desarrollo de software moderno, desde la administración de sistemas hasta la implementación de algoritmos avanzados, pasando por el procesamiento de datos y la creación de interfaces web interactivas. Los desafíos superados y las soluciones implementadas constituyen un valioso bagaje técnico que servirá como fundamento para proyectos futuros de mayor envergadura y complejidad.

AUTORÍA

- Claudia Maria Lopez Bombin
 - Implementacion de código, y
 - Redaccion de memoria.
- Gonzalo Velasco Lucas
 - Implementacion de código, y
 - Redaccion de memoria.